What Is HTML?

Think of a document that you would create in a word processor like Microsoft Word or Google Docs. They usually consist of more than one style.

They use different font sizes to indicate different sections of the text, like headers, body paragraphs, footers, table of contents, captions, and so on.

Unlike humans who can simply look at a document and understand the difference between a heading and a paragraph, computers don't have that intuition. In order to correctly render a web page, it must be explicitly told what each piece of content is.

So how exactly do we instruct the browser to display web content? This is where Hypertext Markup Language (or HTML for short) can come in handy.

HTML is the language that most websites are written with. It is used to create web pages and ensure their functionality.

- **Hypertext** defines the link between the web pages.
- **Markup language** is used to define the text document within a **tag**, which determines the structure of web pages.

HTML allows you to take a plain text document created in any simple text editor and organize it into lists. It also creates links to other web pages, includes images, and much more.

So in other words, HTML outlines the structure or the skeleton of our web page using **tags**.

Just like most programming languages, we type a bunch of HTML into a file (a.k.a. document) so we can send it around.

HTML documents are files that end with a **.html** or **.htm** extension. You can view them using any web browser (such as Google Chrome, Safari, or Mozilla Firefox). The browser reads the HTML file and renders its content so that internet users can view it.

The only thing that we have to do is to change the file extension into **.html**

1. HTML Tree

Since HTML is the skeleton of our web page, it should be very organized and well-structured to make it easy for the browser to display it. Here HTML is represented as a tree of tags. The root element of this tree is always the <html> tag.

2. HTML Code Structure:

As we have seen, HTML is based on tags. (<>)

We have to know that:

- All HTML documents must start with a document type declaration: .
- The HTML document itself begins with < html > and ends with </ html>.
- The visible part of the HTML document is between < body > and .
- A tag is closed this way: </ tagname>. For example, < head > is closed with < /head >.
- Changes are held between tags (For example, the code between < b > and < /b > becomes bold.



Let's Start Coding

- It’s time to get our hands a little bit dirty!

- Remember the folder structure that we have talked about in the previous chapter? It's time to use it.
- Inside the GoMyCode folder, we are going to create a file named **index.html**

1. Open this file in VSCode and paste the following code.

```
<!DOCTYPE html>

<html>

  <head>          <title> Hello world ! </title>     </head>

     <body>

          <h1> This is my first application ! </h1> <!-- this is a comment
-->

  </body>


</html>
```

2. Save your changes.

3. Open **index.html** in the browser and see the result.

- No need to overthink it because we are going to understand things step by step.
- Everything in HTML is built using tags. There are two types of tags:

1. **Paired tag**: it has to be closed, it is composed of an opening tag and a closing tag with content inside of it.
2. **Unpaired tag**: It does not have to be closed because it is a self closing tag.

- Every **element** in our HTML tree is composed of an opening tag <h1> , content, and a closing tag </h1> .
- Comments are very important for developers to understand their own code and other people's code. The comment does not appear to the user. It exists only in the code file. To create a comment in HTML, you only need to put it inside a comment tag <!-- whatever you want to say --> .
- One last thing, the HTML file structure is always the same.

The HTML Tag:

As we have seen earlier, everything is contained inside the < html > tag.

- The HTML tag is a crucial tag. It tells the browser that the content is located between < html > and < /html>.
- It is also called the “root” element.

```
<html>
    <!--Everything-->
</html>
```

The Body Tag:

The body element appears after the head element in the page. It should contain all the content of your web page: text, images, and so on.

All web pages have one single body element, in order to understand the tags inside the body, we will divide them into:

- Headings
- Text Formatting and Line Breaking Tags

- Paragraphs
- Images
- Audio and Video
- Links
- Lists
- Tables
- Buttons
- Forms and Inputs

Let's get to know them one by one.

The Head tag:

```
<head>
 <title>My Beautiful Page</title>
 <link rel="icon" href="favicon.ico" type="image/x-icon"/>
</head>
```

The head element contains information about the web page. You can put many elements inside of it such as: Title => sets the title of the page.

Link => links elements to html, such as the page icon, CSS file, etc. We will see the other tags later on in the course. Here's an example down below, " rel" and "href" are called tag attributes, we will get back to them later in the course. For now, to set the icon, you need to set rel to "icon" and href to the link to your **image.**

Heading Tags:

The HTML standard has five additional text heading elements, appropriately defined from < h1 > to < h6 >. They are considered as Paired Tags (We have to close them).

```
<h1>This is a h1 Heading</h1>
```

This is a h1 Heading

```
<h2>This is a h2 Heading</h2>
```

This is a h2 Heading

```
<h3>This is a h3 Heading</h3>
```

This is a h3 Heading

```
<h4>This is a h4 Heading</h4>
```

This is a h4 Heading

```
<h5>This is a h5 Heading</h5>
```

This is a h5 Heading

```
<h6>This is a h6 Heading</h6>
```

This is a h6 Heading

Paired Tag:

```
This is a <b>bold</b> tag test
```

This is a **bold** tag test

```
This is an <i>italic</i> tag test
```

This is an *italic* tag test

```
This is a <del>deleted</del> tag test
```

This is a ~~deleted~~ tag test

- Paired tags: if the text is placed between a tag and its companion tag. Example : <b>: Bold text <i>: Italic text <small>: Displays smaller text <del>: Defines deleted text
- Unpaired tag: does not have a companion tag. In other words it doesn’t close.
: Breaks a line

Note: HTML doesn’t read spaces, you need to use
 to jump a line.

Paragraph Tag:

The < p > Tag defines a paragraph in HTML. The browser breaks a line without the need of the < br> tag. Paragraphs are automatically separated by a line break. It's similar to writing a paragraph in a word document, paragraphs need to be separated from each other for them to be coherent and understandable.

Here’s an example :

```
<p>My first paragraph</p>

<p>My second paragraph</p>
```

My first paragraph

My second paragraph

HTML Attributes

HTML attributes provide additional information about an HTML element. Attributes can be considered as properties of the element. An element can have a single attribute, many attributes, or no attributes at all. Attributes are placed in the opening tag, with a space after the element declaration (or after another attribute if there are multiple).

Let's take a look at an example heading with a title attribute:

```
<h2 title="This is a subheading">Hello, World!</h2>
```

Here's the output that we should have :

Hello, World!

The Image Tag:

The < img > tag is responsible for adding images to your HTML page. Basically, the “src” attribute takes the URL of your image or its path on your computer. The “alt” attribute is your plan B if your image fails to load. Unlike most of the elements we have encountered thus far, the img element does not have a closing tag.

```
<!-- Incorrect img declaration  -->

<img src="path/to/image/dog.jpg" alt="A dog" > </img>

<!-- Correct img declaration  -->

<img src="path/to/image/dog.jpg" alt="A dog" >
```

Linked Page

One of the most important aspects of the World Wide Web is the ability to connect with other parts of the Web. Without a method to redirect HTML pages to other web addresses, the Web as we know it would cease to exist. We can connect an HTML page to other web pages by creating a hyperlink using the anchor tag.

The a tag:

```
<a href="https://gomycode.tn/">Take me to GoMyCode's Website</a>
```

[Me conduire au site web de GoMyCode](https://gomycode.tn/)

Links allow users to navigate between different web pages. If you click on a link , you can jump to another document. Links are defined with the < a > tag, also called anchor element. The “href” attribute defines the path once it’s clicked on.

Internal Link:

Links can be local too. Local links allow users to navigate on the same web page. In other words, you can click on a link and jump to the end of the page, or to an element on your web page.

```
<body>
    <a name="TopOfThePage"> </a>
    <h1>Link Up Top</h1>
    <p>
```

```
      Lorem, ipsum dolor sit amet consectetur adipisicing elit. Iure in
ducimus,

      tenetur facere repellendus ipsa beatae minus est laudantium nobis
tempora

      suscipit sunt tempore reprehenderit nulla necessitatibus! Eius sed

      provident iusto dicta corrupti itaque, ducimus quo illo cupiditate,

      quaerat blanditiis, quod accusamus doloremque.



      Neque unde nostrum dignissimos ad quam ratione.

    </p>

    <img src="Gomycode.png" alt="Gomycode" />

    <p>

      Lorem ipsum dolor sit amet consectetur adipisicing elit.




      Quia, consequuntur!

    </p>

    <img src="Track.jpg" alt="Gomycode">

    <a href="#TopOfThePage"> Let's go up Top </a>

  </body>
```

Internal Link:

This is what it should look like. As previously mentioned, local links make it easier to navigate in your web pages.



The Audio Tag:

To play an audio HTML file, use the < audio > element: The controls attribute is necessary. It adds audio controls like play, pause, and volume control. The < source > element allows you to specify alternative audio files that the browser can choose from. The browser will use the first recognized format. The text between < audio ><

/audio> will be displayed only if the audio file is not supported by the browser.

```
<audio controls>
    <source src="test.mp3" type="audio/mpeg">
        Please Check the audio Extension
</audio>
```

The Video Tag:

We can also add videos to our web page by using the < video > element: It is preferable to include height and width attributes in your tag. In addition, to automatically start a video we use the autoplay attribute:

```
<video width="320" height="240" autoplay>
      <source src="test.mp4" type="video/mp4" />
      Please Check the video extension.
</video>
```

The Iframe Tag:

We can also add videos from YouTube by simply using the iframe tag. Just go to YouTube, pick a video, go to Share -> Embed -> Copy tag < iframe >

```
<iframe
      width="420"
      height="315"
      src="https://www.youtube.com/embed/zcTFG_F0FRs"
></iframe>
```

HTML Lists :

Lists are used to arrange related pieces of information so they are connected, understandable, and easy to read. In modern web

development, lists are workhorse elements, frequently used for navigation as well as general content.

Lists are advantageous from a structural point of view because they help create a well-structured, highly accessible, and easy-to-maintain document. They are also useful because they provide specialized elements to which you can attach CSS styles. Semantically correct lists help visitors read your website and they simplify maintenance when your pages need to be updated.

There are many types of lists. Let's have a look at the most used ones:

- Ordered list
- Unordered list
- Description list

Unordered List:

To create an unordered list, we use the HTML tag `<ul>` . An unordered list is typically rendered as a bulleted list but we can change the style of the list using the `type` attribute. Every element of the list is wrapped with tag `<li>`

```
<h1>Web Languages</h1>
<ul>
      <li>HTML</li>
      <li>CSS</li>
      <li>JS</li>
</ul>
```

Our code output would be :

Web Languages:

- HTML

- CSS
- JS

We can also change the styling type by using `type` attribute:

```
<h1>Tracks At GoMyCode</h1>


    <!-- Disc bullets -->
    <ul type="disc">
        <li>AI Track</li>
        <li>Web Track</li>
        <li>Game Track</li>
        <li>Data Science Track</li>
    </ul>


    <!-- Square bullets -->
    <ul type="square">
        <li>AI Track</li>
        <li>Web Track</li>
        <li>Game Track</li>
        <li>Data Science Track</li>
    </ul>
```

Ordered List:

The HTML `<ol>` element represents an ordered list of items, typically rendered as a numbered list but, like the unordered list, it can be changed.

```
    <h1>Web Ordered Languages:</h1>

     <ol>

        <li>HTML</li>

        <li>CSS</li>
```

```
        <li>JS</li>

    </ol>
```

Our code output would be :

Web Ordered Languages:

1. HTML
2. CSS
3. JS

Ordered list type values can be used to change the numbering scheme, and include the following:

- 1: Default numeric scheme
- I: Upper-case Roman numerals
- i: Lower-case Roman numerals
- A: Upper-case alphabet

- a: Lower-case alphabet

Here's an example. You can try to do the rest by yourself.

```
<!-- Upper-case Roman Numerals -->

    <ol type="I">

        <li>Create a folder named "GoMyCode" on your Desktop</li>

        <li>Create a new folder in GoMyCode folder name it whatever you
want.</li>

        <li>Create a new folder inside it and name it "res"</li>

        <li>Create a new folder inside it and name it "css"</li>

    </ol>
```

Description List:

The HTML `<dl>` element represents a description list. The element encloses a list of groups that contain terms (specified using the `<dt>` element) and descriptions (provided by `<dd>` elements). The common uses for this element are: implementing a glossary or displaying metadata (a list of key-value pairs).

```
<p>The House Sigils In GOT:</p>


<dl>
	<dt>House Stark</dt>
	<dd>The Direwolf</dd>

	<dt>House Lannister</dt>
	<dd>The Lion</dd>

	<dt>House Targaryen</dt>
	<dd>The Three-Headed Dragon</dd>

	<dt>House Baratheon</dt>
	<dd>The Stag</dd>
</dl>
```

Div Tag:

 is one of the most useful tags in HTML. The , a block-level element, is like a component or a bag. It can contain many HTML elements in order to apply changes to them as a whole section.

 helps in organizing your code as well as improving creativity with CSS. We will learn how it does that later in the course.

```
<div>

    <h1>This is the first division</h1>
```

```
    </div>

    <div>

      <h1>This is the second division</h1>

    </div>
```

Our code output would be :

This is the first division

This is the second division

Table Tag

There are plenty of situations where you'll want to present a table of data on your web page. If you haven't worked with tabular data before, it will be useful to know that a table consists of rows and columns. Each row/column pair has a piece of data corresponding to it, called as a table cell.

Let's dive straight in and convert the table below into HTML.

HTML	CSS	JS
Paragraphs	Selecteurs	DOM
Tables	Styles	Éléments

HTML tables are defined with the < table > tag.

- < th > tags are column names
- < tr > tags are rows
- < td > tags are column values of the row

```
    <table border="1">
    <tr>
        <th>HTML</th>
        <th>CSS</th>
        <th>JS</th>
    </tr>
    <tr>
```

```
        <td>Paragraphs</td>
        <td>Selectors</td>
        <td>DOM</td>
      </tr>
      <tr>
        <td>Tables</td>
        <td>Styles</td>
        <td>Elements</td>
      </tr>
    </table>
```

1. What Is a Semantic Tag?

Up until now, we have focused on using HTML to structure our web pages and provide a clear presentation of the content. Following the HTML5 standard, you need to clearly indicate the meaning of each component of your web page's content, this is what we call semantic HTML.

Focusing on using HTML to semantically structure your web content gives you several advantages:

- Making your web content vastly more accessible to readers with disabilities.
- CSS style application becoming more consistent and predictable.
- Search engines optimizing and better understanding your web pages.

2. Semantic Tags Use:

As we have already mentioned, semantic tags are used to add meaning to our HTML document. A code should be comprehensible by whoever reads it.

Our document is usually divided into three different sections:

< header >

The header is typically a group of introductory or navigational aids. It can contain some elements such as a logo, a search form, a slogan, etc.

< main >

The main section will be explained further in the next slides.

< footer >

The footer typically contains information about the author, contact information, copyright data, etc.

3. Nav Tag:

Now that we have our page divided into header, main, and footer, let's move on to more semantic tags that can be useful to our code.

The <nav> element defines a set of navigation links.

```
<nav>
```

```
<a href="/html/> HTML </a>

<a href="/css/> CSS </a>

<a href="/js/> JavaScript </a>

<a href="/jquery/> jQuery </a>

</nav>
```

4. Section Tag:

The <section> element defines a section in a document.

For example, if we want to split our home page into several sections for introduction, content, contact information, etc.

```
<section>

<h1> Contact information <h1>

<p> Go My Code is an EdTech Startup.. </p>

</section>
```

5. Article Tag:

The <article> element defines an article in a document.

It can mean a blog entry, a news/scholarly article, or a forum post.

```
<article>

<h1> What is the perks of being a Web Developer? <h1>

<!-- Article contents -->

</article>
```

6. Aside Tag:

HTML offers many semantic elements to define distinct parts of a web page.

We'll list the two most used semantic elements :

- The <aside> element : Is like a sidebar. It allows us to navigate through the main content.

- The <figure> and <figcaption> elements : it regroups images and captions in one element.

7. The Advantages of Semantic Tags:

By adding semantic tags to your document you provide extra information about that document which helps in communication. Semantic tags produce the meaning of a page and its content, making it extremely clear to the browser.



1. What Is an HTML Form?

In HTML, forms are a way of receiving input from users and they are very useful for collecting website data. Forms are highly relevant in

today's modern age as they are commonly used for sign-ups, authentication, and comments.

A form can be created with the <form> tag.

```
<html>
 <head>
 </head>
 <body>
   <form>
     A form
   </form>
 </body>
</html>
```

The code output would be : A form

2. A Quick Look inside:

Here's a quick look at what a form would look like using only HTML. Don't mind the tags inside the <form> , we are going to have a closer look at them later.

```
    <form>

        <label>Name:</label>

        <input type="text" name="first name" placeholder="Type your name here" />



        <label>Number:</label>

        <input type="number" name="number" value="23123456" />



        <label>Birth Date:</label>

        <input type="date" name="birthday" />



        <label>Password:</label>
```

```
      <input type="password" name="password" />
```

```
      <input type="submit" value="Submit" />
```

```
    </form>
```

1. Form Input:

The most useful component of a form is the input tag, which creates a text field where users can enter data. Here's an example:

```
<form>
```

```
     Search <input type = "text" name = "search" />
```

```
</form>
```

Output

As seen in the code above, there is a type and name associated with inputs. The type defines the nature of the input (text, URL, email, etc.), while the name allows us to access the input data for future use. You can think of the name as a variable in which the input data is stored. Other properties of input include:

- Size
- Value
- Max length
- Read only

2. Selection Inputs

You can use <select> (with nested <option>) elements to create a drop-down selection of items that a user can choose from.

```
    <select>
        <option value="WebTrack">Web Track</option>
        <option value="AITrack">AI Track</option>
        <option value="GameTrack">Game Track</option>
        <option value="DataScienceTrack">Data Science Track</option>
    </select>
```

3. Text Area:

If you want your user to be able to include new lines (by pressing return) in their text input, you can use a <textarea> element:

- The rows attribute specifies the visible number of lines in a text area.
- The columns (cols) attribute specifies the visible width of a text area.

```
<textarea name="message" cols="30" rows="5">
        Welcome to GoMyCode
</textarea>
```

4. Buttons:

A <button> element should be used whenever you want to create a clickable button to perform a certain action on the page.

<button> elements are simple to define, and have three different type values:

- Submit: submits form data to a server.
- Reset: resets all the data in the current form.
- Button: indicates no default behavior. This type of button will be more useful when we start dealing with Javascript.

```
    <form>
        <label>First Name:</label>
        <input type="text" name="firstName" />

        <label>Last Name:</label>
        <input type="text" name="lastName" id="lastName" />

        <!-- This button will submit the form, causing the page to redirect
-->
        <button type="submit">Submit Name</button>
    </form>
```

What Is CSS?

As we have mentioned, in this chapter we are going to be focusing on style, presentability, and the appearance of our web page. The main language used for styling a web page is CSS.

Fasten your seatbelts because in this chapter we are going to learn:

- What is CSS?
- How do you integrate CSS into HTML?
- How do you select an HTML element?
- How do you change the position of an element?

Cascading Style Sheets, commonly referred to as **CSS**, is a simple design language intended to simplify the process of making web pages presentable.

CSS determines the look and feel of a web page. With **CSS**, you can control the color of a text, the style of fonts, the spacing between paragraphs, and how columns are sized and laid out. It also defines what background images or colors are used, the layout designs, and, most importantly, the variations in display to adapt to different devices and screen sizes.

CSS is easy to learn and understand. It also offers powerful control over the presentation of an HTML document. Most commonly, CSS is combined with the markup language HTML.

1. Linking CSS to HTML:

Before starting with CSS and how it works, we should first know how to link CSS to our HTML code.

There are three possible ways to attach CSS to HTML:

- Inline : by using the style attribute in HTML elements.

- Internal : by using a < style > element in the < head > section.
- External : by using an external CSS file.

2. Inline CSS:

The first way to style our HTML elements is via the attribute style where we can add the design we want to this element.

In situations where we have a small number of elements to deal with, this could be very helpful. But once we're dealing with large pages with lots of moving pieces, it will become extremely tedious to apply a separate style attribute to each element.

```
<body>
    <h1 style="color: aqua">Hello World</h1>
    <p style="border: 1px blue; color: crimson">
        This is my first paragraph
    </p>
</body>
```

3. Internal CSS:

The second way to add the CSS style is to include it directly inside the HTML document. Notice the use of the style HTML element nested in the head element. The style element can be used to place CSS rules inline with an HTML document, like in the example below. Although

this is an absolutely valid way of adding CSS to your web pages, it will not let us reuse the same CSS style in other HTML files.

```
<!DOCTYPE html>

<html lang="">

    <head>

        <meta charset="" />

        <meta name="viewport" content="width=, initial-scale=" />

        <!-- Internal style -->

        <style>

            h1 {

                color: aqua;

                font: sans-serif;

            }

            p {

                color: darkblue;

                font: small;

            }

        </style>

        <title></title>

    </head>

    <body></body>
```

```
</html>
```

4.External CSS

The last and probably the best way to use CSS (in terms of code reuse) is to create an external file with the extension **.css** and import it into our HTML document with the special link tag.

The link tag is placed inside the head of the document, with the attribute href that indicates the path of the CSS file.

```
<!DOCTYPE html>
<html lang="">
    <head>
        <meta charset="" />
        <link rel="stylesheet" href="./styles.css" />
        <meta name="viewport" content="width=, initial-scale=" />
        <title></title>
```

```
    </head>

    <body></body>

</html>
```

1. CSS Selector:

After attaching the CSS style, our next step is figuring out how to attach a chosen style to the proper element. If we want to change the style of a particular element in our HTML page, we have to select that element first. To do so, CSS provides a set of rules (each rule consists of a selector to indicate the element meant for modification) followed by a declaration block that contains a set of properties and those properties′ values.

2. Selector Types:

CSS rules can select an element in a variety of ways. The three basic kinds of selectors are:

1. **Type** selectors: are used to select HTML elements by element name.
2. **Class** selectors: are used to select HTML elements by a specific class value.

3. **Id** selectors: are used to select an HTML element associated with a specific ID value.

There are other types of CSS selectors, but we are not going to cover them right now. You can find out more by following this

link

.

1. Type Selector:

In the previous slides, we have learned how to pick an HTML element in order to style it and we have encountered several types of CSS selectors. Let’s view the first type of selector and implement it in a real example.

Type:

Using a type selector is as simple as typing the name of the element. However, it's preferable if you know when to use it. This selector is used when we want to apply the same rule on every element in the page.

For example, we want every h1 title to be in red, the font will be roboto and the text-align will be in the center.

- HTML

```
<!DOCTYPE html>
<html lang="">
    <head>
        <title>Type selector</title>
    </head>
    <body>
        <h1>Welcome to my blog</h1>
        <p>
```

```
                Lorem ipsum dolor sit amet consectetur adipisicing elit.
Blanditiis

                ratione alias, eaque temporibus tenetur similique
accusamus quas

                laudantium ea reiciendis itaque atque earum provident. Ab
nisi sunt quo

                repellat vitae.

            </p>

            <h1>I am a super web developer</h1>

        </body>

</html>
```

- CSS

```
h1 {

    color: red;

    font-family: roboto;

    text-align: center;

}
```

- Output:

2. Class Selectors:

The selection by type has worked fine until now, but let's say we have shared style not only for one type but for multiple types. For example, we are going to put all the text in a blue section and the other section

will be in gray. In this case, the type selector is not extremely helpful. That is why we instead chose to use the **class** selector.

A class is a tag attribute that allows us to name an identifier for the element to adopt a certain style. You can specify a class on CSS with a (.) period

To use the class selector, you can follow the example below:

- HTML

```
   <!DOCTYPE html>
<html lang="">
      <head>
            <title>Type selector</title>
      </head>
      <body>
            <h1 class="title">Welcome to my blog</h1>
            <p class="title">
                  Lorem ipsum dolor sit amet consectetur adipisicing elit.
Blanditiis
```

```
                ratione alias, eaque temporibus tenetur similique
accusamus quas

            </p>

            <h1>I am a super web developer</h1>

        </body>

</html>
```

- CSS

```
.title{

  color:blue

}
```

3. ID Selector:

The final type of CSS selector is the selection by ID.

ID selectors in CSS allow you to target elements (Tags) by their ID values. ID selectors are unique, so you can only apply them to the

content of one element. To reference an ID, you precede the ID name with a hash mark (#).

- HTML

```
    <h1 id="blue-bordered">Now we are using an Id Selector</h1>
```

- CSS

```
#blue-bordered {
    border: 5px dashed blue;
}
```

- Output:

4. Multiple Selectors :

One of the most important principles in programming is the “DRY” principle: Don’t repeat yourself.

Let's assume that we have a p tag, a div tag, and a section tag that share the same background color. So, by following the “DRY” principle, we must not repeat the styling three times.

CSS allows us to do that. We can select multiple elements by separating the selectors with commas, like this:

```
/* Selecting multiple HTML element types */
        h1, p{
            border: 1px solid black;
        }

      /* Selecting styles to be applied to several classes */
      .FirstItem, .LastItem {
            font-size: 1.2em;
```

```
    }

    /* Using multiple Kinds of selectors */
    h3, .red, #redElement {
        color: red
    }
```

1. Descendant Combinator:

The descendant combinator is typically represented by a single space character. It combines two selectors so that elements matched by the second selector are selected if they have an ancestor (parent, parent's parent, parent's parent's parent, etc.) element matching the first selector. Selectors that utilize a descendant combinator are called descendant selectors.

- HTML

```
<div class="box"><p>Text in .box</p></div>

<p>Text not in .box</p>
```

- CSS

```
            .box p {

                        color: red;

            }
```

- Output

2.Child Combinator:

The child combinator (>) is placed between two CSS selectors. It only matches those elements matched by the second selector that are the direct children of elements matched by the first. Descendent elements further down the hierarchy don't match. For example, to select only <p> elements that are direct children of <article> elements:

- HTML

```
<article>
    <h1>A heading</h1>
    <p>I am a paragraph.</p>
    <div>I am a div</div>
    <p>I am another paragraph.</p>
</article>
```

Css

```
article > p {
    font-weight: bold;
    background-color: #333;
```

```
    color: #fff;

    padding: .5em;

}
```

- Output

3. Adjacent Sibling Combinator:

The adjacent sibling selector (+) is used to select something if it is right next to another element at the same level of the hierarchy. For

example, to select all <p> elements that come right after <h1> elements:

- **HTML** :

```
<article>
    <h1>A heading</h1>
    <p>Veggies es bonus vobis, proinde vos postulo essum magis kohlrabi welsh onion daikon amaranth tatsoi tomatillo
            melon azuki bean garlic.</p>

    <p>Gumbo beet greens corn soko endive gumbo gourd. Parsley shallot courgette tatsoi pea sprouts fava bean collard
            greens dandelion okra wakame tomato. Dandelion cucumber earthnut pea peanut soko zucchini.</p>
</article>
```

- **CSS**:

```
h1 + p {
    font-weight: bold;
    background-color: #333;
```

```
    color: #fff;
    padding: .5em;
}
```

4. General Sibling Combinator

If you want to select siblings of an element even if they are not directly adjacent, then you can use the general sibling combinator (~). To select all <p> elements that come anywhere after <h1> elements, we'd do this:

HTML

```
<article>
    <h1>A heading</h1>
    <p>I am a paragraph.</p>
    <div>I am a div</div>
    <p>I am another paragraph.</p>
</article>
```

CSS

```
h1 ~ p {
    font-weight: bold;
    background-color:red;
    color: #fff;
    padding: .5em;
}
```

Output

1. CSS Pseudo classes:

Have you ever tried to put your cursor on a button and its color changed? Or have you ever clicked on a link and the same thing happened? We call this “marked as clicked” and it can be done with CSS pseudo-classes.

As you can see, the structure is clear. You name your SELECTOR, add : then define your action. Next, we’ll learn a few awesome actions you can do using CSS pseudo-classes. Try them on your editor.

```
    /* unvisited link */
    a:link {
        color: tomato;
    }
    /* visited link */
    a:visited {
        color: tomato;
    }
    /* mouse over element */
```

```
    div:hover {
        color: tomato;
    }
    /* selected link */
    a:active{
        color: #0000FF
    }
```

1. CSS Box Model

Every element in web design is a rectangular box (even if it is a circle).

The CSS box model is defined by the four layers below:

- Content: Is where text and images appear.
- Padding: Is transparent. It clears area around the content.

- Border: Goes around the padding and content. We have already seen it earlier.
- Margin: Clears an area outside the border. The margin is transparent.

Also, we can determine the dimension of any html element through the width and the height

We'll get to know them more individually later.

2. Padding

- We use the padding property to create spacing between an element's content area and border. The padding property applies this spacing in different ways depending on how many values you provide. The Padding is the INSIDE of the element. Suppose we have a div with many HTML elements inside. The problem is,

we want to move the elements inside without moving the div itself. How can we accomplish that? Padding is our hero here!

- We can treat the padding value separately as we have seen in the previous example, or we can make it in one line like the following code.

```
.box {
  padding: <padding-top> || <padding-right> || <padding-bottom> || <padding-left>
}
```

3. Margin

As a first step, go back to the CSS box model scheme to see where the margin is positioned.

Basically, margins allow us to move our tag wherever we want, in any direction: top, bottom, right, or left. The Margin is the OUTSIDE of the element. The margin property is very similar to the padding property, except it allows you to define the spacing around the outside of an HTML element past the border. Like padding, it allows you to define single or multiple values.

4. Height & Width

As we have seen, every element in HTML is a rectangular box. By the laws of geometry, every rectangle is characterized by its width and height.

We can alter these two properties of any HTML content element by using the CSS attribute width or height.

One last thing to always keep in mind: Total_width= width+padding_right+padding_left+margin_right+margin_left To change the value of these properties, just follow the following example:

- HTML

```
    <div>
        <p>I want to get bigger</p>
    </div>
```

- CSS

```
div{
    height: 150px;
    width: 90%;
}
```

5. Border

Like its name implies, the border CSS property sets the border of an element.The border property is a shorthand syntax in CSS that accepts multiple values for drawing a line around the element it is applied to.

```
border:  <border-width> || <border-style> || <color>
```

- border-width: specifies the thickness of the border.
    - : A numeric value measured in px, em, rem, vh and vw units.
    - thin: the equivalent of 1px
    - medium: the equivalent of 3px
    - thick: the equivalent of 5px
- border-style: Specifies the type of line drawn around the element, including:
    - solid: a solid, continuous line.
    - none (default): No line is drawn.

    - hidden: a line is drawn, but not visible. This can be useful for adding a little extra width to an element without displaying a border.
    - dashed: a line that consists of dashes.
    - dotted: a line that consists of dots.
    - double: adds two lines around the element
    - groove: adds a bevel based on the color value in a way that makes the element appear pressed into the document.
    - idge: similar to groove, but reverses the color values in a way that makes the element appear raised.
    - inset: adds a split tone to the line that makes the element appear slightly depressed.
        - outset: similar to inset, but reverses the colors in a way that makes the element appear slightly raised.
- color: specifies the color of the border and accepts <rgb()>, <rgba()>, <code><hsl()>, <hsla()>, <hex-color>, <named-color>
- CSS

```
.box-1{border:none;}
```

```
.box-2{border:5px hidden red;}

.box-3{border:5px solid orange;}

.box-4{border:5px dashed orange;}

.box-5{border:5px dotted orange;}

.box-6{border:5px double orange;}

.box-7{border:5px groove orange;}

.box-8{border:5px ridge orange;}

.box-9{border:5px inset orange;}

.box-10{border:5px outset orange;}
```

- Output:

6. Border Radius

We can give any element “rounded corners” by applying a border-radius through CSS. You’ll only notice if there is a color change involved. For instance, if the element has a background-color or border that is different from the element, it’s incorrect. Simple examples:

- HTML

```
    <div>
        <p>Straight corners</p>
        <p class="rounded">Rounded corners</p>
        <p class="elliptical">Elliptical corners</p>
    </div>
```

- CSS

```
    p {
```

```
        border: 10px solid black;
        margin: 20px;
    }

    .rounded {
        border-radius: 15px;
    }

    .elliptical {
        border-radius: 50px / 25px;
    }
```

- Output:

1. Font Styling:

To style the font, CSS gives us a large set of properties. We'll be studying a few of them:

Font-family

It helps us change the font family of an HTML element, we set one or more fonts for this property .

```
font-family: Helvetica neue, roboto;
```

Color

It changes the text color, it accepts a named color, hexadecimal value, and rgb value.

```
color: rgb(0, 0, 255);

  /* same as

  color: blue;

  color: #0000ff;

  color: #00f;

  color: hsl(0, 100%, 50%); */
```

Font-style

It's used to turn italic text on and off. Possible values are as follows (you'll rarely use this, unless you want to turn some italic styling off for some reason).

```
font-style: bold;

/*

other possible value

 font-style:normal;

 font-style:italic;

 font-style:oblique;

*/
```

Font-weight It sets how bold the text is. This has many values available in case you have many font variants available (such as -light, -normal, -bold, -extrabold, -black, etc.). Realistically, you'll rarely use any of them except for normal and bold.

```
font-weight: normal;

/*

other possible value

 font-weight:bold;

 font-weight:bolder;

 font-weight:lighter;

 font-weight:100-900

*/
```

Text-decoration

It sets or unsets text decorations on fonts (you'll mainly use this to unset the default underline on links when styling them). Available values are:

```
text-decoration: none;

/*

other possible value

 text-decoration:underline;

 text-decoration:overline;

 text-decoration:line-through;

*/
```

2. Text Layout Styles:

With basic font properties out of the way, let's now have a look at properties we can use to affect text layout.

The text-align It's used to control how a text is aligned within its containing content box. The available values are as follows, and work

in pretty much the same way as they do in a regular word processor application: Left: Left- aligns the text Right: Right- aligns the text Center: Centers the text Justify: Makes the text spread out

Line-height It sets the height of each line of text — this includes most length and size units, but can also take a unitless value, which acts as a multiplier and is generally considered the best option. The font-size is multiplied to get the line-height.

Letter-spacing and word-spacing They allow you to set the spacing between letters and words in your text. You won't use these very often, but might find a use for them to get a certain look, or to improve the legibility of a particularly dense font. They can use most length and size units.

CSS Output :

```
first-line {
  letter-spacing: 4px;
  word-spacing: 4px;
```

```
}
```

The code output would be :

3. Text Size:

There will be many instances where you will want to change the default size of text elements. The size of your text can be changed using the font-size property. The font-size takes both absolute and relative values. The most common absolute value is px and the most common relative values are ems and rems. em and rem units are both

relative measurement values that work similarly to percentages. They serve as a multiplier in reference to some other unit of measurement. For font-size:

- 1 em is equivalent to the font-size of the element's parent.
- 1 rem is equivalent to the font-size of the root element of the entire HTML document.

```
h1{
font-size: 50px
}
```

CSS Display:

Every element on a web page is a rectangular box. The display property in CSS determines just how that rectangular box behaves.

The CSS display is most useful when :

- we need to align two HTML elements together in the same line.
- we want to display an HTML element as a block.
- we want to hide an HTML element.

In this lesson, we will see 4 types of CSS display: none, inline, block and inline-block.

Note: The default display of an element is inline or block.

Display: none

We can hide elements by declaring a display: none value. Another way is to declare visibility: hidden instead of display: none, but there is a difference between them..

To show the difference, let's hide one of the boxes below:

- CSS

```
#box-2 {
  display: none;
  width: 100px;
  height: 100px;
  background: blue;
}
```

- Output

- CSS

```
#box-2 {
  width: 100px;
  height: 100px;
  background: blue;
  visibility: hidden;
}
```

- Output

Display: inline

Like the name indicates, this feature enables you to put specified HTML elements in the same line. Let's understand it with an example.

- HTML

```
    <p>One</p>
    <p>Two</p>
    <p>Three</p>
```

- CSS

```
    p {
        display: inline;
    }
```

- Output

One Two Three

1. Display: block

- HTML elements are divided into default block elements (automatically force a line-break without the use of
) and inline elements (need
 to line-break).
- Block-level elements:
    - take full-width (100% width) by default.
    - each get displayed in a new line.
    - allow width & height properties to be set.
    - can contain other block or inline elements.

What about forcing an inline element to act like a block?

- CSS

```
div{
display: block;
}
    p {
```

```
  height: 100px;
  width: 100px;
  background: red;
  color: white;
}
```

- Output

2. Display: inline-block

IIn some cases, both of the display values may not be enough for better web design. In that case, a third display behavior, display: inline-block, comes to the rescue and makes alignment much easier. As we can understand from its name, display: inline-block declaration shows both the characteristics of inline and block-level elements. In other words, we can think of an inline element with width & height properties that can be set, or we can think of a block-level element that doesn't have to start with a new line.

- HTML

```
    <div class="test"></div>

     <div class="test"></div>
```

- CSS

```
      .test {

          width: 50px;

          height: 50px;
```

```
        border: 1px solid red;

        display: inline-block;

    }
```

- Output

1. SAAS

SASS (Syntactically Awesome Style Sheets) is a **CSS preprocessor** that extends the capabilities of CSS by adding features such as variables, nested rules, and mixins. It is a scripting language that is then compiled into CSS. This allows for more maintainable and organized CSS code, as well as the ability to use programming concepts in styling a website. SASS was first released in 2006 and has since become one of the most popular CSS preprocessors, along with its main alternative, Less. One of the main advantages of using SASS is that it can make your CSS more modular, reusable, and easy to maintain. Additionally, SASS supports mathematical operations and

functions, making it easier to handle responsive design and perform calculations in the styling.

In order to write and run SASS code, we need to install a VSCode extension called Live SASS compiler. This is because the browser only reads CSS code and cannot read CSS code hence, we need to install this extension because it is responsible for transforming SASS code into CSS code. In order to write SASS code the extension of our file needs to be .scss.

2. SASS Variables

One of the most common problems in CSS is redundancy to choose colors for all elements, and changing a specific color will lead to changing this color multiple times for other elements, that is why SASS variables were made, where you could define a specific color for example and us this variable for different elements, rather than choosing the color over and over again. In the following example, whenever I want to change the paragraphs text color I can change only the variable $textColor, rather than changing the color in the p tag.

SASS Variables

```
$primaryColor: #58e3e3;

$textColor:black;

#btn1{

    background: $primaryColor;

}

#btn2{

    background: $primaryColor;

}

p{

    color: $textColor;
```

```
}
```

3. Nesting in SASS

The second use for SASS is **nesting**, and this is done by writing CSS-like code but nested inside each other, just like HTML. For example:

Nesting in SASS

```
$hoverColor:rgb(48, 202, 219);

.header{

    // CSS for elements in the header class

    display: flex;

    justify-content: center;

    align-items: center;

    // CSS for buttons inside the header class

    button{

        background-color: $primaryColor;
```

```
        text-align: center;
        &:hover{
            background:$hoverColor;
        }
    }
}
```

4. Separate Code

Another powerful use for SASS is **separating code**, where you don't have to include all your CSS code in one file, you could have separate files for each component or class, and you could import it into other .scss stylesheets. For example we want to write our code for the header class in a separate file, so we call the file _header.scss, and then importing it by writing @import 'header'.

_header.scss file

```
$hoverColor:rgb(48, 202, 219);
```

```
.header{
    // CSS for elements in the header class
    display: flex;
    justify-content: center;
    align-items: center;
    button{
        background-color: $primaryColor;
        text-align: center;
        &:hover{
            background:$hoverColor;
        }
    }
}
```

_variables.scss

```
$primaryColor: #58e3e3;
$textColor:black;
```

styles.scss

```
@import 'header';

@import 'variables';
```

5. Mixins

Mixins are the functions for SASS in other words and are reusable and called anywhere in the code, for example we can make a mixin that centralizes a flexbox, the mixin is going to look something like this:

Mixin code

```
@mixin flexCenter(){
    display: flex;
    justify-content: center;
    align-items: center;
}
// Including the mixin in code
.container{
    @include flexCenter();
}
```

In this example, the only difference is that we included an argument in the mixin, which is the $direction, to allow the user to specify the flex-direction whenever he is using the mixin to make things more flexible.

Mixin code with arguments

```
@mixin flexCenter($direction){
    display: flex;
    justify-content: center;
    align-items: center;
    flex-direction: $direction;
}


// Including the mixin in code
.container{
    @include flexCenter(column);
}
```

To develop more your Idea about SAAS, have a look at

this video

.

CSS: Flexbox

Picking the right display context and managing the layout of a web page can be extremely inconvenient for a web developer. However, it's part of the job and we have to face it head on.

Luckily, the flexbox is the right tool for overcoming this obstacle and is the subject of this skill. Here's an overview of the content to come:

- CSS Flexbox
- Display flex

- CSS position
- CSS grid

Flexbox

Time to meet Flex!

As we have already seen, we can display HTML elements using layout modes: block for sections in a web page, inline for text, etc.

Flexbox will make it easier to design flexible, dynamic, and responsive layout structures without using float or positioning.

Learning Flexbox can be difficult for most people as it's not as enjoyable as other skills we will learn. But, it challenges you to rethink your approach when it comes to layouts in CSS.

Don't worry, we will walk you through everything that you need to know. So, let's jump right in!

What Is Flexbox?

Flexbox has an overly technical definition. For the sake of simplifying this course, we'll skip the technicalities.

You want to imagine Flexbox as the layout ninja for CSS. It's incredibly practical, dynamic, and has a lot of advantages.

When you need to deal with the layout in your styles, the CSS Flexbox model is likely to be your best bet.

How does it work?

Flexbox starts by making a parent element into a flex container.

Use display Flex

How can we access the Flexbox’s super powers?

Pretty simple, let's assume the parent element in question is a div with a class name of flexy, we simply apply the following:

This one line of code enables a flex context for all its direct children. So if all items need to shrink, each one will shrink equally and if there's no space they will be allowed to shrink horizontally until they all fit.

Why Do We Use Flexbox and What Are Its Super Powers?

After we have made the parent a flex container, we can proceed to using it. One of the many things we can do with Flexbox is to center a

child element within a parent. This centering will be along both sides, vertical and horizontal, all according to the main axis.

Columns and Rows

By default, flex items are laid out in their source order. Think of flex items as primarily laid out either in horizontal rows or vertical

columns. Therefore, if we want to have rows or columns, we apply the following rules:

Justify-content

Justify-content defines how elements are aligned along the main axis.

It helps distribute extra free space when either all the flex items on a line are inflexible or when they’re flexible but have reached their maximum size.

Flex-wrap:

One of flex’s properties is that all items will automatically try to fit into one line. You can change that and allow the items to wrap as

needed. So we have two choices: either we decide for them to be in one line or move to another line.

Align-items:

Align-items determines the default behavior for how flex items are laid out along the cross axis on the current line. Think of it as the justify-content version for the cross axis.

Align-content:

What if now we have multiple lines of content and we want to align them? Piece of cake! We naturally use the align-content property.

Keep in mind, this property has no effect when there is only one line of flex items.

Static Position

The **static** position is the default positioning for all the elements on a web page.

The elements are fixed objects within the scene until you decide for them to be a part of a positioning context.

Static means elements are static according to the normal flow of the page. If one moves, another will move as well.

Position Absolute

By default, all HTML elements are in a static position. The element is not positioned in any special way and is always positioned according to the normal flow of the page. In this case the parent is given

position: static, and the child is given position: absolute and according to that, it will be absolute to the parent element that has a position of relative, absolute, or fixed. If there is no position specified, it will be absolute to the page (meaning that it won't be affected by anything around it).

Position Relative

As we have already said, elements are statically positioned by default.

In this case, the parent is given a value of position:relative, and the child is given a value of position:absolute.

When you style an element with position: relative, it creates a positioning context for every child element within the element. Using this positioning you can go ahead and accurately position the child element with respect to any sides of the reference object. Relatively

positioned elements behave exactly like static ones, but they serve as a local frame of reference for absolutely positioned child elements.

Position Fixed

If we position our element as **fixed**, it means the position will always be locked to the size of the browser's window. If you scroll up ,down ,left or right, the element will stay in its position.

Position Sticky

An element with position: sticky is positioned based on the user's scroll position. In other words, as soon as the scroll reaches the sticky element, it will stick to the screen.

CSS Grid System

Every website or application layout you make (or have seen) is essentially boxes placed within certain defined boundaries.

In very simple terms, a grid is just horizontal and vertical lines that define the placement of other design elements.

In the context of the CSS Grid layout, a grid container is a parent that contains all items within it. The grid container defines the initial placement of the grid lines, both vertical and horizontal.

So How Can We Define a Grid?

Just like Flexbox, everything begins with a one-line **display:grid** or **display:inline-grid** for an inline version.

For example, to make a certain div a grid container,

- **CSS**

```
    div {
        display: grid;
    }
```

Columns and Rows

First we have to define columns and rows, because a grid without them is entirely pointless.

To create columns and rows within a grid container you use these two new CSS properties: `grid-template-columns` and `grid-template-rows`.

So how do you use them? Pretty simple.

`grid-template-columns` determines the placement of the columns while `grid-template-rows` determines the placement of the rows within the grid container.

All we need to do is type in the length of these properties,

- CSS

```
    .grid-container {
        display: grid;
        grid-template-columns: 100px 200px 300px;
        grid-template-rows: 100px 200px 300px;
    }
```

Properties of the Grid Items

We have: grid-column-start/grid-row-start the line where the item begins grid-column-end/grid-row-end the line where the item ends

Justify-self Properties

Justify-self aligns a grid item inside a cell along the inline. It can have the value`center`,`end`,`start` or `stretch`.

Time to Practice :

Learn CSS By Playing Games 👾

Learning CSS can be a frustrating experience sometime. One of the best things about learning to code is that people who are good at coding also happen to have the skills to make games that teach other people to code.

FLEXBOX FROGGY

```
 This game is about Flexbox and it covers flex properties like
align-items, justify-content, align-content, flex-direction, align-self,
flex-wrap, flex-flow and order in 24 different levels.
```

Yeah, last level is bit challenging 😉

Grid Garden

Grid Garden game features 28 levels, starting with easy difficulty and gradually increasing it. Use the various grid properties to water and grow your carrot garden. Grid Garden has a code editor and visual area so you instantly see your result.

CSS Diner

The CSS Diner webapp is really fun and genuinely educational for developers of all skill levels. You'll go through 26 different levels of

increasing difficulty. The first few levels teach core fundamentals like element selectors and class selectors. In the later levels you're working with advanced concepts like :only-of-type and :not(x).

What's responsive design?

Responsive web design is an *approach* whereby a designer creates a web page that *"responds to"* or resizes itself depending on the type of device it is being seen through. That could be an oversized desktop computer monitor, a laptop or devices with small screens such as smartphones and tablets.

Responsive Web design has become an essential tool for anyone with a digital presence. With the growth of smartphones, tablets and other mobile computing devices, more people are using smaller-screens to view web pages.



How to make it responsive?

Responsive design is accomplished through CSS `media queries`. Think of media queries as a way to conditionally apply CSS rules. They tell the browser that it should ignore or apply certain rules depending on the user's device.

What Is Media Query?

Media queries are at the heart of responsive design. They let you target specific screen sizes and specify CSS rules that will be executed on that screen.

The most popular way that media queries are used is something called the @media rule.

- CSS

```
@media screen and (max-width: 300px) {
    /* write your CSS in this code block */
}
```

What Does It Really Do?

So to translate the code below, it means “For a screen device with a maximum width of 300px … implement the following actions” Any styles within the code block will only apply to devices that match the

expression, screen, and max-width (300px) i.e. screen devices with a maximum width of 300px.

- CSS

```
@media screen and (max-width: 300px) {
    /* write your CSS in this code block */
}
```

Media Query Breakpoints

What is a breakpoint? CSS breakpoints are points where the website content responds according to the device width, allowing you to show the best possible layout to the user.

These are the most used breakpoints:

```
/* Extra small devices */
```

```
    @media (max-width: 576px) { /* ... */ }

/* Small devices (landscape phones, 576px and up)  */

    @media (min-width: 576px) and (max-width: 768px) { /* ... */ }

/* Medium devices (tablets, 768px and up)  */

    @media (min-width: 768px) and (max-width: 992px)  { /* ... */ }

/* Large devices (desktops, 992px and up)  */

    @media (min-width: 992px) and (max-width: 1200px)  { /* ... */ }

/* Extra large devices (large desktops, 1200px and up)  */

    @media (min-width: 1200px) { /* ... */ }
```

Font-Awesome

Now, we will be sharing with you some helpful and useful resources to make your web page look “awesome”. Time to meet Font-Awesome! It is the most popular way to add font icons to your website. Font Awesome icons are created using scalable vectors, so you can use high quality icons that work well on any screen size.

How Do We Use It?

First, we add the following line inside the < head > section of your HTML page:

```
<link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/font-awesome/4.7.0/css/font-awesome.min.css">
```

Then, to display cool icons we simply use the material available on

https://fontawesome.com/

. You will find everything you will need on that website. Here's an example:

- HTML

```
<i class="fa fa-car"></i>

<i class="fa fa-car" style="font-size:48px"></i>

<i class="fa fa-car" style="font-size:60px; color:blue"></i>
```

- Output

Font-face

Let's get to know another useful tool, it's called the font-face rule.

With `font-face` , web developers or designers have the freedom to use new and interesting fonts, and throw the classic and dull fonts right out the window.

It's very easy to use the font-face rule. First you must define a name for the font (example: Chathura Bold). Then, point it to the font file (Src: url(../Path to your font file).

And then you simply use it as a normal font in your CSS file.

```
        @font-face {

                font-family: "chathurabold";

                src:
url("../font/chathura_bold_macroman/chachathura-bold-webfont.eot");
```

```
            font-weight: normal;

            font-style: normal;

        }
```

What Is Bootstrap?

Remember when we spoke about responsive design? Well, it's time to meet Bootstrap. It will be your responsive design guide.

How will Bootstrap make your life easier?

Bootstrap code is already written and is available for everyone to use. So, we'll be treating it like a library and borrow only what we need from it. Everything we need is already on the Internet. We just have to be selective, thorough, and pick out only what we need. This is called "snippeting".

Snippeting is basically copying code from another source and pasting it to your code. It is ethical and unproblematic in Bootstrap, no need to worry.

How Is It Used?

We will now look at a few of the many examples of how to use Bootstrap. You can find more information on Bootstrap's official website. Feel free to try new things! Like every library, to have access

to its elements you must add the following line inside the section of your HTML page:

```
<link rel="stylesheet"
href="https://maxcdn.bootstrapcdn.com/bootstrap/3.4.1/css/bootstrap.min.css">
```

Bootstrap Grid System

One of Bootstrap's most important is that it makes your website responsive using its grid system.

The main idea is to divide the screen into twelve equal columns, and this gives us a new unit that works independently from the size of the screen.

There are four types of resolutions (or classes) in Bootstrap:

- xs (for phones - screens less than 768px wide)
- sm (for tablets - screens equal to or greater than 768px wide)
- md (for small laptops - screens equal to or greater than 992px wide)
- lg (for laptops and desktops - screens equal to or greater than 1200px wide)

How do you use it?

First Step

To apply Bootstrap's grid system to our web page we need to use Bootstrap's container and rows. First thing we need to do is create a div with `class="container"`.It is as essential as the body tag when we're working in Bootstrap.

So, our Bootstrap must be inside the div with the class container. It makes a margin on the right and on the left so that our content is centered.

If we want our page to take up the whole screen, we write `container-fluid`.

```
<body>
    <div class="container">
        <!-- Some BootStrap Classes here -->
    </div>
</body>
```

How do you use it?

The Second Step

After creating the container, we can add rows. Inserting a row, like the following picture demonstrates, means a line across the screen. That row will be divided into twelve equal parts where we can put our HTML elements and that will replace the margin and padding.

- HTML

```
    <div class="container-fluid">
        <div class="row" style="background-color:lightblue">
            First row
        </div>
        <hr />
        <div class="row" style="background-color:#e5cf0d">
            Second row
        </div>
        <hr />
        <div class="row" style="background-color:#b30de5">
            Third row
        </div>
```

```
    </div>
```

- Output

How Does It Work?

So, how does the grid system work?

Each row that we add can be divided into 12 equal parts called "columns".

Since the row is divided into 12 columns, if an element gets 12 columns of width it will be as wide as the whole screen. If it takes 6,

then it's as wide as half the screen. Here's an example, col-8 will take 8 columns, and col-4 will take 4 columns.

- HTML

```
<div class="row">
    <div class="col-8">col-8</div>
    <div class="col-4">col-4</div>
</div>
```

- Output

Bootstrap Classes

As we have already mentioned, Bootstrap is a massive collection of reusable and versatile pieces of code which are written in CSS, HTML, and JavaScript. With that said, developers have a range of components and tools at their disposal, which can be used on the website. These include, but are not limited to:

- Drop-down menus
- Navigation bars
- Progress bars
- Thumbnail images

Let’s take a look at some of these free tools.

Buttons

So in HTML, buttons are basic, dull, and uninteresting. They also don't have a meaning behind them. Why not improve them and make them more awesome and more advanced with Bootstrap?

- HTML

```
<button type="button" class="btn btn-primary">Primary</button>
<button type="button" class="btn btn-secondary">Secondary</button>
<button type="button" class="btn btn-success">Success</button>
<button type="button" class="btn btn-danger">Danger</button>
<button type="button" class="btn btn-warning">Warning</button>
<button type="button" class="btn btn-info">Info</button>
<button type="button" class="btn btn-light">Light</button>
<button type="button" class="btn btn-dark">Dark</button>
<button type="button" class="btn btn-link">Link</button>
```

- Output

Alert

Alerts can be used in different situations. Let's introduce some of them. You can imagine each scene yourself.

- HTML

```
<div class="alert alert-primary" role="alert">
  This is a primary alert—check it out!
</div>
<div class="alert alert-secondary" role="alert">
  This is a secondary alert—check it out!
</div>
<div class="alert alert-success" role="alert">
  This is a success alert—check it out!
</div>
<div class="alert alert-danger" role="alert">
```

```
  This is a danger alert—check it out!

</div>

<div class="alert alert-warning" role="alert">

  This is a warning alert—check it out!

</div>
```

- Output

Navbar

Bootstrap also offers a large range of navigation bars to make things easier. A navigation bar is a navigation header placed at the top of the page. Here's an example: (You will find the needed code in the Bootstrap's official website)

Progress Bar

Another useful feature is a progress bar. They can be used for creating your resume page. For example:

- HTML

```
<div class="progress">
  <div class="progress-bar" role="progressbar" style="width: 25%;"
aria-valuenow="25" aria-valuemin="0" aria-valuemax="100">25%</div>
```

```
</div>
```

- Output